# DAY 1
# PYTHON FOUNDATIONS FOR AI

From Python Beginner to AI-Ready Programmer

| | |
|---|---|
| **Course** | AI & Data Science Bootcamp - Day 1 |
| **Covers** | Python Basics, Control Flow, Data Structures, Functions, Error Handling & Hands-on Projects |
| **Format** | Concept + Example + Code + Output + Real-World Use + Practice |

A complete, example-driven guide covering every topic in the Day 1 syllabus - built for learners moving from zero programming experience toward Python skills used in Artificial Intelligence, Machine Learning, Data Science and Automation.

# TABLE OF CONTENTS


| | | |
|---|---|---|
| **1.** | Introduction to Python | 4 |
| **2.** | Python Fundamentals - Variables, Data Types & Operators | 8 |
| **3.** | Input, Output & Control Flow | 16 |
| **4.** | Data Structures - Lists, Tuples, Sets & Dictionaries | 24 |
| **5.** | Functions & Error Handling | 32 |
| **6.** | Hands-On Project 1 - Student Performance Analyzer | 37 |
| **7.** | Hands-On Project 2 - Smart Expense Calculator | 40 |
| **8.** | Day 1 Challenge - Student Result Management System | 43 |
| **9.** | Recap, Cheat Sheet & Self-Check Quiz | 46 |


**HOW TO USE THIS GUIDE**

Page numbers are approximate section markers within this guide. Every topic from the official Day 1 syllabus is covered in full, in the same order it appears in your curriculum, with extra worked examples added for clarity.

DAY 1 | CHAPTER 1

# Introduction to Python

## 1.1 What is Python?

Python is a high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. "High-level" means Python code reads almost like plain English and hides the complicated details of memory management and hardware instructions that lower-level languages like C expose to the programmer. This makes Python an excellent first language, and also the most popular language for advanced fields such as Artificial Intelligence (AI), Machine Learning (ML), and Data Science.

Python is **interpreted** (code is run line-by-line by a program called the Python interpreter, not compiled into machine code ahead of time), **dynamically typed** (you do not need to declare the type of a variable in advance), and supports multiple programming styles including procedural, object-oriented and functional programming.

### Why is Python so widely used?

| Domain | Why Python Fits |
|---|---|
| Artificial Intelligence & ML | Libraries such as TensorFlow, PyTorch and scikit-learn are built with Python-friendly interfaces, letting researchers describe complex models in a few readable lines. |
| Data Science | Pandas, NumPy and Matplotlib make cleaning, analysing and visualising data fast and intuitive - a single line can process a million-row spreadsheet. |
| Automation / Scripting | Python scripts can rename thousands of files, send emails, scrape websites, or fill spreadsheets automatically, saving hours of manual work. |
| Web Development | Frameworks like Django and Flask let developers build complete websites and REST APIs quickly. |
| Beginner Friendliness | Simple, English-like syntax (no semicolons, no complex braces) means new programmers can focus on logic instead of fighting the language. |

**REAL-WORLD CONNECTION**

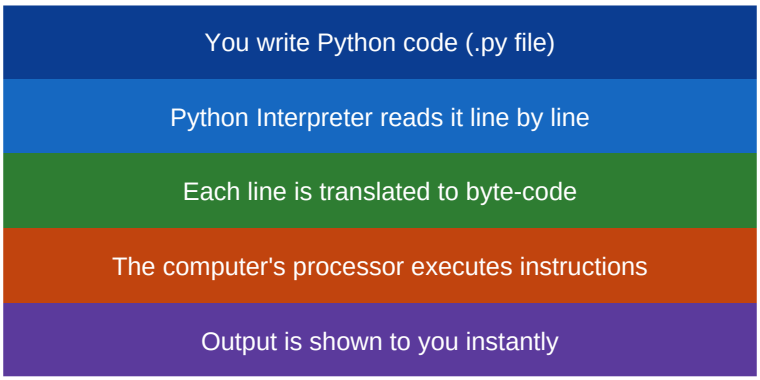
> Think of Python as the "universal remote control" of programming languages. Just as one remote can operate your TV, set-top box and sound system, Python can drive a chatbot (AI), clean a company's sales data (Data Science), rename 10,000 photos on your laptop (Automation) and run a shopping website (Web Development) - all with the same core language.

### Key Features of Python

- **Easy to Read and Write:** Uses indentation instead of curly braces, so code visually mirrors its logic.
- **Free and Open Source:** Anyone can download, use and contribute to Python at no cost.
- **Huge Standard Library:** "Batteries included" - math, file handling, dates, networking and more are built in.

- **Massive Ecosystem:** Hundreds of thousands of third-party packages (NumPy, Pandas, Django, Flask, TensorFlow) are installable in seconds via `pip`.
- **Cross-Platform:** The same code runs on Windows, macOS and Linux without modification.
- **Strong Community Support:** Millions of developers worldwide means answers to almost any problem are one search away.

### 1.2 How Python Code Actually Runs




**WHY THIS MATTERS**

> Because Python is interpreted rather than compiled, you get immediate feedback - write a line, run it, see the result. This tight feedback loop is one reason Python is loved for quick experimentation, such as testing an AI model idea before building the full system.

## 1.3 Setting Up Python: Google Colab vs VS Code

There are two very common ways to write and run Python for AI and Data Science learning. Both are used throughout this course.

| Feature | Google Colab | VS Code (Visual Studio Code) |
|---|---|---|
| Installation | None - runs fully in your browser | Requires installing Python + VS Code on your computer |
| Internet Needed? | Yes, always | No, works fully offline once installed |
| Best For | Quick experiments, AI/ML notebooks, sharing code easily | Full projects, larger applications, professional development |
| Hardware | Free access to Google's GPUs/TPUs (useful for AI) | Uses your own computer's hardware |
| File Format | Notebook cells (.ipynb) | Plain script files (.py) |

### Running Python in Google Colab

1. Go to `colab.research.google.com` in your browser and sign in with a Google account.
2. Click **New Notebook**. A page with an empty grey "cell" appears.
3. Type Python code into the cell, e.g. `print("Hello, AI World!")`.
4. Press **Shift + Enter** (or click the play button) to run that cell. The output appears directly below it.

## Running Python in VS Code

1. Install Python from `python.org` (tick "Add Python to PATH" during setup on Windows).
2. Install VS Code from `code.visualstudio.com`.
3. Open VS Code, install the official **Python extension** from the Extensions panel.
4. Create a new file named e.g. `day1.py` and write your code.
5. Open the integrated terminal (`Ctrl + ``) and run:

**CODE**

```
python day1.py
```

**OUTPUT / EXPLANATION**

```
Hello, AI World!
```

## Your First Python Program

**CODE**

```
# This is a comment - Python ignores this line when running
print("Hello, AI World!")   # print() displays text on the screen
print("Welcome to Day 1 of Python Foundations for AI")
```

**OUTPUT / EXPLANATION**

```
Hello, AI World!
Welcome to Day 1 of Python Foundations for AI
```

Notice two things: first, the line starting with # is a **comment** - notes for humans that Python skips entirely. Second, `print()` is a **function**: a reusable block of behaviour that takes input (the text in quotes) and produces output (text shown on screen). You will use `print()` constantly throughout this course to inspect what your programs are doing - including later, when debugging AI models.

**COMMON PITFALL**

> Common beginner mistakes at this stage: forgetting the closing quotation mark, forgetting the closing parenthesis, or mismatching indentation. Python is strict about indentation - unlike many languages, it uses indentation (spaces) to define which lines belong together, so keep it consistent (4 spaces is the standard convention).

## Quick Practice 1.1

1. Open Google Colab (or VS Code) and print your own name and course batch using two separate `print()` statements.
2. Add a comment above each line explaining what it does.
3. Try removing a closing quote intentionally and observe the error message Python shows - this builds familiarity with reading errors, a critical AI/ML debugging skill.

DAY 1 | CHAPTER 2

# Python Fundamentals: Variables, Data Types & Operators

## 2.1 Variables

A **variable** is a named container that stores a value in the computer's memory so you can reuse or change it later. In Python, you create a variable simply by assigning a value to a name using the = sign - there is no need to declare its type in advance.

**CODE**

```
student_name = "Ananya"      # a piece of text (string)
student_age = 20              # a whole number (int)
attendance_percent = 92.5     # a decimal number (float)
is_passed = True              # True/False value (boolean)

print(student_name, student_age, attendance_percent, is_passed)
```

**OUTPUT / EXPLANATION**

```
Ananya 20 92.5 True
```

### Variable Naming Rules

- Must start with a letter or an underscore (_), never a digit.
- Can contain letters, digits and underscores only - no spaces or symbols like `@ - %`.
- Case-sensitive: `Marks` and `marks` are different variables.
- Cannot be a reserved Python keyword such as `if`, `for`, or `class`.
- By convention, use lowercase words separated by underscores, e.g. `total_marks` (this style is called *snake_case*).

**REAL-WORLD CONNECTION**

A variable is like a labelled box in a warehouse. The label (variable name) does not change, but you can open the box and replace its contents (the value) at any time - swap out yesterday's inventory count for today's without renaming the box itself.

## 2.2 Data Types

Every value in Python has a **type**, which determines what operations are valid on it and how it behaves. The four basic types on Day 1 are:

| Type | Keyword | Example | Real-World Analogy |
|---|---|---|---|
| Integer | int | 25, -7, 1000 | A whole count - number of students in a class |

| Type | Keyword | Example | Real-World Analogy |
|---|---|---|---|
| Floating Point | float | 92.5, 3.14, -0.5 | A measured quantity - height in metres, percentage |
| String | str | "Hello", "AI2026" | A piece of text - a name, an address, a sentence |
| Boolean | bool | True, False | A yes/no switch - is the light on? Did the student pass? |
| None Type | NoneType | None | An empty box - deliberately "no value yet" |

**CODE**

```
age = 21
print(type(age))          # <class 'int'>

price = 499.99
print(type(price))        # <class 'float'>

city = "Tirupati"
print(type(city))         # <class 'str'>

is_registered = False
print(type(is_registered))# <class 'bool'>

result = None
print(type(result))       # <class 'NoneType'>
```

**OUTPUT / EXPLANATION**

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
```

The built-in `type()` function is one of the most useful debugging tools you will ever use - when a program misbehaves, checking a variable's type is often the fastest way to find the bug. This becomes essential later when a dataset column that should hold numbers accidentally contains text.

**NOTE**

`None` is not the same as 0, an empty string, or False. It specifically means "no value has been assigned" - for example, a student's grade before it has been calculated.

## 2.3 Operators

Operators are special symbols that perform actions on values (called **operands**). Python groups them into several families.

### 2.3.1 Arithmetic Operators

| Operator | Meaning | Example | Result |
|---|---|---|---|
| + | Addition | 7 + 3 | 10 |
| - | Subtraction | 7 - 3 | 4 |

| Operator | Meaning | Example | Result |
|---|---|---|---|
| * | Multiplication | 7 * 3 | 21 |
| / | Division (always returns float) | 7 / 2 | 3.5 |
| // | Floor Division (drops the remainder) | 7 // 2 | 3 |
| % | Modulus (remainder) | 7 % 2 | 1 |
| ** | Exponent (power) | 2 ** 3 | 8 |

**CODE**

```
marks1, marks2, marks3 = 78, 85, 91
total = marks1 + marks2 + marks3
average = total / 3

print("Total:", total)
print("Average:", average)
print("Is total exactly divisible by 3?", total % 3 == 0)
```

**OUTPUT / EXPLANATION**

```
Total: 254
Average: 84.66666666666667
Is total exactly divisible by 3? False
```

**REAL-WORLD CONNECTION**

The modulus operator (%) is exactly how a bank determines your change, how a calendar app finds the day of the week, and how AI systems split data into batches - "give me the leftover after dividing into equal groups."

## 2.3.2 Comparison Operators

Comparison operators compare two values and always produce a Boolean result (`True` or `False`).

| Operator | Meaning | Example | Result |
|---|---|---|---|
| == | Equal to | 10 == 10 | True |
| != | Not equal to | 10 != 5 | True |
| > | Greater than | 10 > 5 | True |
| < | Less than | 10 < 5 | False |
| >= | Greater than or equal to | 10 >= 10 | True |
| <= | Less than or equal to | 9 <= 10 | True |

**COMMON PITFALL**

A very common beginner error is confusing = (assignment - "store this value") with == (comparison - "are these equal?"). `if marks = 90:` is invalid Python; it must be `if marks == 90:`.

## 2.3.3 Logical Operators

Logical operators combine multiple Boolean expressions into one, essential for writing real decision rules such as eligibility checks.

| Operator | Meaning | Example | Result |
|---|---|---|---|
| and | True only if BOTH sides are True | (80 > 50) and (90 > 50) | True |
| or | True if AT LEAST ONE side is True | (30 > 50) or (90 > 50) | True |
| not | Reverses a Boolean value | not (10 > 5) | False |

**CODE**

```
attendance = 85
internal_marks = 40

eligible_for_exam = (attendance >= 75) and (internal_marks >= 30)
print("Eligible for final exam:", eligible_for_exam)
```

**OUTPUT / EXPLANATION**

```
Eligible for final exam: True
```

## 2.3.4 Assignment Operators

Assignment operators store a value in a variable, and the shorthand versions update a variable based on its own current value - extremely common when accumulating totals in loops.

| Operator | Equivalent To | Example (starting x = 10) | New Value of x |
|---|---|---|---|
| = | assign | x = 10 | 10 |
| += | x = x + value | x += 5 | 15 |
| -= | x = x - value | x -= 3 | 7 |
| *= | x = x * value | x *= 2 | 20 |
| /= | x = x / value | x /= 4 | 2.5 |

## 2.3.5 Membership Operators

`in` and `not in` check whether a value exists inside a sequence such as a string, list or tuple (covered fully in Chapter 4).

**CODE**

```
subjects = ["Python", "Maths", "AI Basics", "Statistics"]

print("AI Basics" in subjects)        # True
print("Chemistry" in subjects)        # False
print("Chemistry" not in subjects)    # True

word = "Tirupati"
print("pat" in word)                  # True - substring check
```

**OUTPUT / EXPLANATION**

```
True
False
True
True
```

### 2.3.6 Identity Operators

`is` and `is not` check whether two variables point to the exact same object in memory - not just equal values. This is subtly different from ==.

**CODE**

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b)   # True  - same content
print(a is b)   # False - different objects in memory
print(a is c)   # True  - c points to the SAME object as a
```

**OUTPUT / EXPLANATION**

```
True
False
True
```

**TIP**

> Use == to compare values ("do these hold the same data?") and `is` only to compare identity ("are these literally the same object?"), most commonly used as `if x is None:`.

## 2.4 Input and Output

So far our programs have used fixed values written directly in the code. Real programs need to interact with the user - the `input()` function reads what the user types, and `print()` displays results.

**CODE**

```
name = input("Enter your name: ")
age = input("Enter your age: ")

print("Hello,", name + "!")
print("Next year you will be", int(age) + 1, "years old.")
```

**OUTPUT / EXPLANATION**

```
Enter your name: Rahul
Enter your age: 20
Hello, Rahul!
Next year you will be 21 years old.
```

**IMPORTANT**

`input()` ALWAYS returns a string, even if the user types a number. That is why the code above wraps it with `int(age)` before adding 1 - adding an integer to a string directly would raise a `TypeError`. This conversion step, called **type casting**, is one of the most important habits to build early.

## Formatted Output with f-strings

Python's f-strings (formatted string literals) are the modern, readable way to build output messages that mix text and variables - written constantly in AI/ML scripts to report results.

**CODE**

```
name = "Sneha"
marks = 88.5
grade = "A"

print(f"{name} scored {marks} marks and secured grade {grade}.")
print(f"Rounded score: {marks:.0f}")     # format to 0 decimal places
```

**OUTPUT / EXPLANATION**

```
Sneha scored 88.5 marks and secured grade A.
Rounded score: 88
```

## Quick Practice 2.1

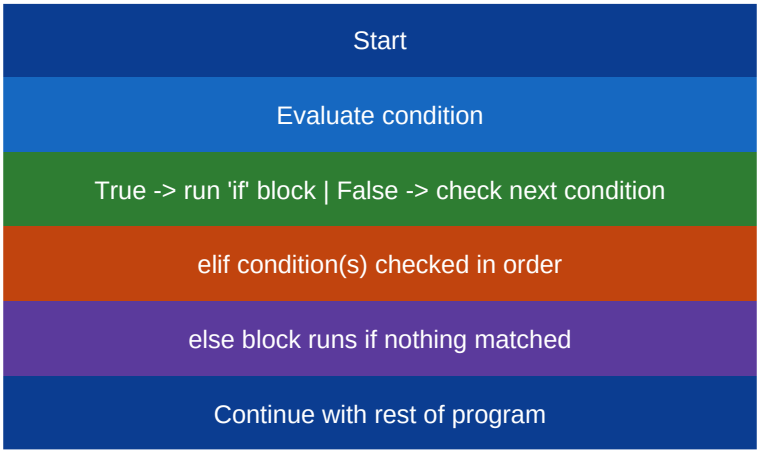
1. Create variables for a product's name (str), price (float), quantity (int) and whether it is in stock (bool). Print all four using an f-string in one readable sentence.
2. Take two numbers from the user with `input()`, convert them to `int`, and print their sum, difference, product and quotient.
3. Using logical operators, write an expression that checks if a person can vote AND has a valid ID.

# Control Flow: Decisions & Loops

## 3.1 if, elif, else

Control flow statements let a program make decisions and choose which block of code to execute. The `if` statement runs a block only when a condition is `True`.

### Decision-Making Flow




**CODE**

```
marks = 76

if marks >= 90:
    print("Grade: A+")
elif marks >= 75:
    print("Grade: A")
elif marks >= 60:
    print("Grade: B")
else:
    print("Grade: F - Needs Improvement")
```

**OUTPUT / EXPLANATION**

```
Grade: A
```

Python checks conditions from top to bottom and stops at the **first** one that is True. Since 76 is not ≥ 90 but is ≥ 75, the second branch runs and every later branch is skipped entirely - this efficiency matters when a program checks thousands of records.

### Nested Conditions

A nested condition is an `if` statement placed inside another `if` block, used when a decision depends on more than one independent question.

**CODE**

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Eligible to vote.")
    else:
        print("Age OK, but valid ID required.")
else:
    print("Not eligible - under 18.")
```

**OUTPUT / EXPLANATION**

```
Eligible to vote.
```

**REAL-WORLD CONNECTION**

Nested conditions mirror an airport security check: first "Is the passenger's boarding pass valid?" (outer if) - only if yes does the officer even check "Does the photo ID match?" (inner if). If the boarding pass fails, the ID is never checked at all.

**COMMON PITFALL**

Python decides where a block starts and ends purely through **indentation** (4 spaces is standard). Mixing tabs and spaces, or misaligning indentation, causes an `IndentationError` - one of the very first errors every Python learner encounters.

## 3.2 Loops: for and while

Loops let you repeat a block of code multiple times without rewriting it - essential for processing lists of students, rows of data, or training an AI model over many iterations.

### 3.2.1 The for Loop

A `for` loop iterates over a sequence (a string, list, tuple, or range of numbers), running the loop body once for each item.

**CODE**

```
subjects = ["Python", "Maths", "AI Basics"]

for subject in subjects:
    print("Studying:", subject)
```

**OUTPUT / EXPLANATION**

```
Studying: Python
Studying: Maths
Studying: AI Basics
```

### 3.2.2 range()

`range()` generates a sequence of numbers and is most often used to repeat an action a fixed number of times.

| Call | Produces | Meaning |
|---|---|---|
| range(5) | 0, 1, 2, 3, 4 | Start at 0, stop BEFORE 5 |
| range(2, 6) | 2, 3, 4, 5 | Start at 2, stop BEFORE 6 |
| range(0, 10, 2) | 0, 2, 4, 6, 8 | Start at 0, stop before 10, step by 2 |
| range(10, 0, -1) | 10, 9, ... , 1 | Counting downward with a negative step |

**CODE**

```
for i in range(1, 6):
    print("Attempt number", i)
```

**OUTPUT / EXPLANATION**

```
Attempt number 1
Attempt number 2
Attempt number 3
Attempt number 4
Attempt number 5
```

### 3.2.3 The while Loop

A `while` loop repeats **as long as** a condition remains True. It is the right choice when you don't know in advance how many repetitions are needed - for example, "keep asking the user until they enter a valid password."

**CODE**

```
attempts = 0
password = ""

while password != "python123" and attempts < 3:
    password = input("Enter password: ")
    attempts += 1

if password == "python123":
    print("Access granted.")
else:
    print("Access denied - too many attempts.")
```

**OUTPUT / EXPLANATION**

```
Enter password: hello
Enter password: 12345
Enter password: python123
Access granted.
```

**IMPORTANT**

> Every `while` loop needs something that eventually makes its condition False - here, either the correct password is entered or `attempts` reaches 3. Forgetting this creates an **infinite loop** that never stops, a classic beginner bug.

### 3.2.4 break and continue

`break` exits a loop immediately, skipping any remaining iterations. `continue` skips just the current iteration and moves to the next one.

**CODE**

```
# break example - stop searching once found
scores = [45, 67, 89, 92, 55]
for score in scores:
    if score > 90:
        print("First score above 90 found:", score)
        break

# continue example - skip failing scores, only print passing ones
for score in scores:
    if score < 50:
        continue
    print("Passing score:", score)
```

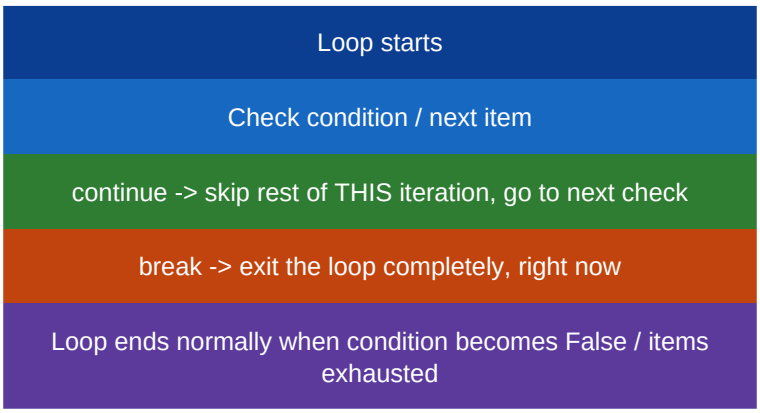
**OUTPUT / EXPLANATION**

```
First score above 90 found: 92
Passing score: 67
Passing score: 89
Passing score: 92
Passing score: 55
```

**REAL-WORLD CONNECTION**

Think of a security guard scanning a queue of visitors for one specific VIP badge: once found, they stop scanning (`break`). Now imagine a bouncer checking a guest list and simply waving through anyone not on a banned list without further checks - they `continue` to the next guest instead of stopping entirely.

## break vs continue at a glance




## Quick Practice 3.1

1. Write a program that prints all even numbers from 1 to 50 using a `for` loop and `range()`.
2. Write a `while` loop that keeps adding numbers entered by the user until they type 0, then prints the running total.
3. Using nested `if` statements, classify a triangle as equilateral, isosceles or scalene given three side lengths entered by the user.

DAY 1 | CHAPTER 4

# Data Structures: Lists, Tuples, Sets & Dictionaries

Data structures are ways of organising multiple values together under a single variable name. Choosing the right structure is one of the most important skills in programming, and directly determines how efficiently an AI or Data Science pipeline can store and process information.

| Structure | Ordered? | Changeable? | Duplicates Allowed? | Written As |
|---|---|---|---|---|
| List | Yes | Yes (mutable) | Yes | [1, 2, 3] |
| Tuple | Yes | No (immutable) | Yes | (1, 2, 3) |
| Set | No | Yes (mutable) | No | {1, 2, 3} |
| Dictionary | Yes (insertion order) | Yes (mutable) | Keys: No, Values: Yes | {"a": 1} |

## 4.1 Lists

A **list** is an ordered, changeable collection that can hold items of any type, including mixed types, and is written with square brackets.

**CODE**

```
marks = [78, 85, 91, 66, 72]
print(marks)
print("First mark:", marks[0])       # indexing starts at 0
print("Last mark:", marks[-1])       # negative index counts from the end
print("Number of marks:", len(marks))
```

**OUTPUT / EXPLANATION**

```
[78, 85, 91, 66, 72]
First mark: 78
Last mark: 72
Number of marks: 5
```

### Common List Operations

**CODE**

```
marks = [78, 85, 91, 66, 72]

marks.append(95)          # add to the end
print("After append:", marks)

marks.remove(66)           # remove a specific value
print("After remove:", marks)

marks.sort()               # sort ascending, in place
print("Sorted:", marks)

marks.sort(reverse=True)    # sort descending
print("Descending:", marks)

print("Slice (first 3):", marks[0:3])   # slicing
print("Sum:", sum(marks), " Max:", max(marks), " Min:", min(marks))
```

**OUTPUT / EXPLANATION**

```
After append: [78, 85, 91, 66, 72, 95]
After remove: [78, 85, 91, 72, 95]
Sorted: [72, 78, 85, 91, 95]
Descending: [95, 91, 85, 78, 72]
Slice (first 3): [95, 91, 85]
Sum: 421  Max: 95  Min: 72
```

| Method / Function | Purpose |
|---|---|
| append(x) | Adds x to the end of the list |
| insert(i, x) | Inserts x at position i |
| remove(x) | Removes the first occurrence of value x |
| pop(i) | Removes and returns the item at index i (last item if i omitted) |
| sort() / sort(reverse=True) | Sorts the list ascending / descending, in place |
| len(list) | Number of items in the list |
| sum(list), max(list), min(list) | Total, largest and smallest numeric value |

**REAL-WORLD CONNECTION**

A list is like a class attendance register: names are stored in a specific order, you can add a new student at the end, remove one who left, and the same name could theoretically appear twice if unmanaged - exactly the flexible, ordered, editable behaviour a list provides.

## 4.2 Tuples

A **tuple** looks similar to a list but uses round brackets and, crucially, is **immutable** - once created, its contents cannot be changed. Tuples are used for data that should never change during the program, such as fixed coordinates or a database record.

**CODE**

```
student_record = ("Ananya", 20, "CSE-AIDS")
print(student_record)
print("Name:", student_record[0])

try:
    student_record[1] = 21     # this will fail - tuples are immutable
except TypeError as e:
    print("Error:", e)
```

**OUTPUT / EXPLANATION**

```
('Ananya', 20, 'CSE-AIDS')
Name: Ananya
Error: 'tuple' object does not support item assignment
```

**WHEN TO USE**

Use a tuple when the data represents a fixed record that should not accidentally be modified later in the program - for example, the (latitude, longitude) of a fixed location, or the (r, g, b) values of a colour.

## 4.3 Sets

A **set** is an unordered collection of **unique** values, written with curly braces. Sets automatically remove duplicates and are extremely fast for membership testing ("is this value already present?") and for finding unions/intersections between groups.

**CODE**

```
branch_a = {"Ravi", "Sneha", "Karan", "Sneha"}   # duplicate "Sneha" is dropped automaticall
branch_b = {"Karan", "Meera", "Divya"}

print("Branch A students:", branch_a)
print("Students in BOTH branches:", branch_a & branch_b)     # intersection
print("Students in EITHER branch:", branch_a | branch_b)     # union
print("Only in Branch A:", branch_a - branch_b)               # difference
```

**OUTPUT / EXPLANATION**

```
Branch A students: {'Ravi', 'Sneha', 'Karan'}
Students in BOTH branches: {'Karan'}
Students in EITHER branch: {'Ravi', 'Sneha', 'Karan', 'Meera', 'Divya'}
Only in Branch A: {'Ravi', 'Sneha'}
```

**NOTE**

Because sets are unordered, printing one may show items in a different sequence than they were added, and you cannot access items by index (`branch_a[0]` is invalid).

**REAL-WORLD CONNECTION**

A set behaves like a guest list at the door of an event: it doesn't matter what order names were added, and no name can be checked in twice - the doorman only cares whether a name IS or ISN'T on the list, which is exactly the fast lookup a set is built for.

## 4.4 Dictionaries

A **dictionary** stores data as **key-value pairs**, written with curly braces, letting you look up a value instantly using a meaningful key instead of a numeric index. This is arguably the most -used data structure in real Data Science work, mirroring how a single row of a spreadsheet or a JSON API response is represented in Python.

**CODE**

```
student = {
    "name": "Ravi Teja",
    "roll_no": "21CSE045",
    "marks": [78, 85, 91],
    "is_passed": True
}

print(student["name"])
print(student["marks"])

student["attendance"] = 92          # add a new key-value pair
student["roll_no"] = "21CSE045-A"   # update an existing value

for key, value in student.items():
    print(key, "->", value)
```

**OUTPUT / EXPLANATION**

```
Ravi Teja
[78, 85, 91]
name -> Ravi Teja
roll_no -> 21CSE045-A
marks -> [78, 85, 91]
is_passed -> True
attendance -> 92
```

| Method | Purpose |
|---|---|
| dict.keys() | Returns all keys |
| dict.values() | Returns all values |
| dict.items() | Returns all (key, value) pairs together |
| dict.get(key, default) | Safely fetch a value; returns default instead of an error if missing |
| dict[key] = value | Add a new pair, or update an existing key's value |
| del dict[key] | Remove a key-value pair |

**REAL-WORLD CONNECTION**

A dictionary is exactly like a real paper dictionary or a phone contacts app: you look up a word (key) to instantly get its meaning (value), or a contact's name (key) to instantly get their phone number (value) - you never have to search item-by-item from the start.

## 4.5 Practical Data Manipulation - Bringing It Together

Real programs combine these structures. A very common pattern is a **list of dictionaries** - exactly how a spreadsheet, a database table, or an AI training dataset is often represented in Python.

**CODE**

```
students = [
    {"name": "Ananya", "marks": 91},
    {"name": "Ravi",   "marks": 67},
    {"name": "Sneha",  "marks": 45},
]

total = 0
topper = students[0]

for s in students:
    total += s["marks"]
    if s["marks"] > topper["marks"]:
        topper = s

average = total / len(students)
print(f"Class average: {average:.2f}")
print(f"Topper: {topper['name']} with {topper['marks']} marks")
```

**OUTPUT / EXPLANATION**

```
Class average: 67.67
Topper: Ananya with 91 marks
```

This single example already resembles a miniature analytics pipeline: it loops through structured records, accumulates a total, and tracks a running "best" value - the same logical pattern used later to find the best-performing model in a machine learning experiment.

## Quick Practice 4.1

1. Create a list of 5 city names, sort it alphabetically, then print only the first 3 cities using slicing.
2. Create a dictionary representing a product (name, price, quantity). Add a new key `"total_value"` equal to price times quantity.
3. Given two sets of hobbies belonging to two friends, print the hobbies they have in common and the hobbies unique to each friend.

# Functions & Error Handling

## 5.1 Defining Functions

A **function** is a named, reusable block of code that performs a specific task. Functions let you write logic once and reuse it anywhere, avoiding repetition and making programs easier to test and maintain - a principle called **DRY** (Don't Repeat Yourself).

**CODE**

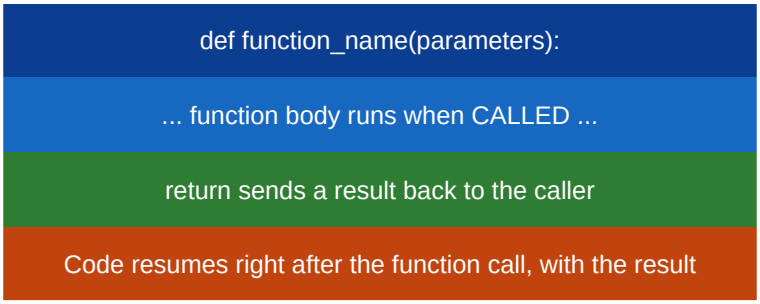
```
def calculate_percentage(obtained, total):
    """Returns the percentage score given obtained and total marks."""
    percentage = (obtained / total) * 100
    return percentage

result = calculate_percentage(450, 500)
print(f"Percentage: {result}%")
```

**OUTPUT / EXPLANATION**

```
Percentage: 90.0%
```

### Anatomy of a Function




| Term | Meaning |
|---|---|
| Definition | The def block that describes what the function does (written once) |
| Parameter | A placeholder name listed inside the parentheses of the definition |
| Argument | The actual value passed in when the function is called |
| Return value | The result sent back using the return keyword |
| Call | Actually running the function by writing its name with () |

## 5.2 Default Arguments

A parameter can have a **default value**, used automatically when the caller does not supply that argument - useful for optional settings.

**CODE**

```
def greet_student(name, course="AI & Data Science"):
    print(f"Welcome {name} to the {course} program!")

greet_student("Divya")                      # uses the default course
greet_student("Karan", "Full Stack Web Dev")  # overrides the default
```

**OUTPUT / EXPLANATION**

```
Welcome Divya to the AI & Data Science program!
Welcome Karan to the Full Stack Web Dev program!
```

## 5.3 Lambda Functions

A **lambda** is a small, anonymous (unnamed) function written in a single line, typically used for short throwaway operations, especially with functions like `sorted()`, `map()` and `filter()`.

**CODE**

```
square = lambda x: x ** 2
print(square(5))

students = [("Ravi", 67), ("Ananya", 91), ("Sneha", 45)]
students_sorted = sorted(students, key=lambda s: s[1], reverse=True)
print(students_sorted)
```

**OUTPUT / EXPLANATION**

```
25
[('Ananya', 91), ('Ravi', 67), ('Sneha', 45)]
```

**TIP**

> A lambda is equivalent to a normal function but written compactly: `lambda x: x ** 2` means the same thing as defining `def square(x): return x ** 2`. Use lambdas only for very short, one-line logic; anything more complex should be a full `def` function for readability.

## 5.4 Variable Scope

**Scope** determines where a variable can be seen and used. A variable created inside a function (a **local** variable) only exists while that function runs and is invisible outside it. A variable created outside any function (a **global** variable) can be read from anywhere.

**CODE**

```
course_name = "AI & Data Science"    # global variable

def show_batch_info():
    batch_year = 2026                # local variable - only exists inside this function
    print(f"{course_name} - Batch {batch_year}")

show_batch_info()
print(course_name)     # works - global variable is visible everywhere

try:
    print(batch_year)   # fails - local variable does not exist outside the function
except NameError as e:
    print("Error:", e)
```

**OUTPUT / EXPLANATION**

```
AI & Data Science - Batch 2026
AI & Data Science
Error: name 'batch_year' is not defined
```

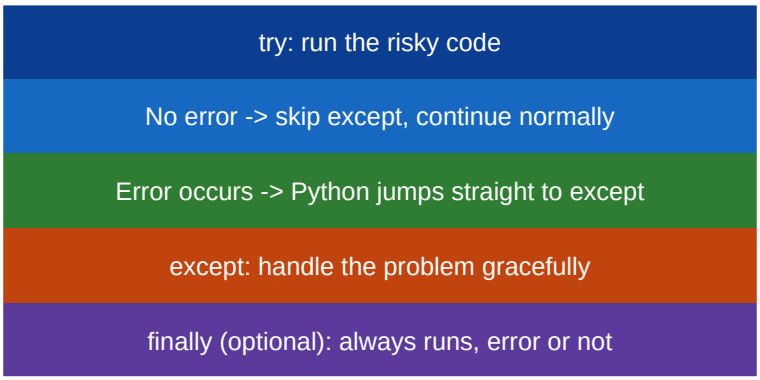
**REAL-WORLD CONNECTION**

Think of global variables as an announcement on a college notice board - visible to everyone in the building. A local variable is like a note scribbled on a student's personal notepad during one exam - useful only during that exam, then gone; other students (other functions) never see it.

## 5.5 Error Handling: try / except

Even correct-looking programs can fail at runtime due to bad input, missing files, or unexpected values - these are called **exceptions**. Python lets you anticipate and gracefully handle them using `try` / `except` instead of letting the whole program crash.

### try / except Flow



**CODE**

```
def get_marks_input():
    try:
        marks = int(input("Enter marks (0-100): "))
        if marks < 0 or marks > 100:
            raise ValueError("Marks must be between 0 and 100")
        return marks
    except ValueError as e:
        print("Invalid input:", e)
        return None

marks = get_marks_input()
if marks is not None:
    print("Marks accepted:", marks)
else:
    print("Please try again with a valid number.")
```

**OUTPUT / EXPLANATION**

```
Enter marks (0-100): abc
Invalid input: invalid literal for int() with base 10: 'abc'
Please try again with a valid number.
```

| Common Exception | Typical Cause |
|---|---|
| ValueError | Converting an invalid string to a number, e.g. int("abc") |
| TypeError | Mixing incompatible types, e.g. "5" + 5 |
| ZeroDivisionError | Dividing any number by zero |
| IndexError | Accessing a list position that does not exist |
| KeyError | Accessing a dictionary key that does not exist |
| NameError | Using a variable that was never defined (or is out of scope) |

**CODE**

```
try:
    total_marks = 250
    num_subjects = 0
    average = total_marks / num_subjects
except ZeroDivisionError:
    print("Cannot calculate average - number of subjects is zero!")
finally:
    print("Marks-processing attempt finished.")
```

**OUTPUT / EXPLANATION**

```
Cannot calculate average - number of subjects is zero!
Marks-processing attempt finished.
```

**NOTE**

> The `finally` block runs no matter what - whether an error happened or not - and is typically used for cleanup actions such as closing a file or a database connection.

## Quick Practice 5.1

1. Write a function `is_prime(n)` that returns True or False for whether n is a prime number.
2. Write a function that safely divides two numbers entered by the user, using try/except to handle both invalid input and division by zero.
3. Write a lambda function that converts a temperature from Celsius to Fahrenheit, then use it inside a loop to convert a list of five temperatures.

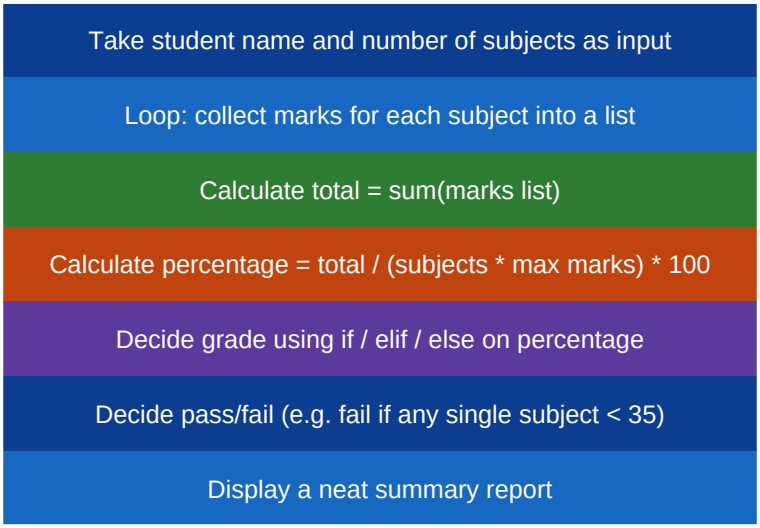
# Hands-On Project 1: Student Performance Analyzer

## 6.1 Problem Statement

Build a program that accepts a student's name and marks in several subjects, then calculates the total, percentage, grade, and pass/fail status - combining variables, data types, operators, control flow, lists and functions from every section of Day 1 into one working application.

## 6.2 Designing the Logic

### Program Flow




| Percentage Range | Grade |
|---|---|
| 90 and above | A+ |
| 75 - 89.99 | A |
| 60 - 74.99 | B |
| 50 - 59.99 | C |
| 35 - 49.99 | D |
| Below 35 | F (Fail) |

## 6.3 Full Program

**CODE**

```
def calculate_grade(percentage):
    """Return a letter grade for a given percentage."""
    if percentage >= 90:
        return "A+"
    elif percentage >= 75:
        return "A"
    elif percentage >= 60:
        return "B"
    elif percentage >= 50:
        return "C"
    elif percentage >= 35:
        return "D"
    else:
        return "F"


def student_performance_analyzer():
    name = input("Enter student name: ")
    num_subjects = int(input("Enter number of subjects: "))
    max_marks_per_subject = 100
```

**CODE**

```
    marks_list = []
    for i in range(1, num_subjects + 1):
        marks = int(input(f"Enter marks for Subject {i} (out of 100): "))
        marks_list.append(marks)

    total = sum(marks_list)
    max_total = num_subjects * max_marks_per_subject
    percentage = (total / max_total) * 100
    grade = calculate_grade(percentage)

    # Fail condition: any subject below 35, regardless of overall percentage
    has_failed_subject = any(m < 35 for m in marks_list)
    status = "FAIL" if has_failed_subject or percentage < 35 else "PASS"
```

**CODE**

```
    print("\n----- STUDENT PERFORMANCE REPORT -----")
    print(f"Name           : {name}")
    print(f"Marks Entered  : {marks_list}")
    print(f"Total Marks    : {total} / {max_total}")
    print(f"Percentage     : {percentage:.2f}%")
    print(f"Grade          : {grade}")
    print(f"Result         : {status}")
    print("---------------------------------------")


student_performance_analyzer()
```

**OUTPUT / EXPLANATION**

```
Enter student name: Ananya Reddy
Enter number of subjects: 4
Enter marks for Subject 1 (out of 100): 88
Enter marks for Subject 2 (out of 100): 92
Enter marks for Subject 3 (out of 100): 79
Enter marks for Subject 4 (out of 100): 95

----- STUDENT PERFORMANCE REPORT -----
Name           : Ananya Reddy
Marks Entered  : [88, 92, 79, 95]
Total Marks    : 354 / 400
Percentage     : 88.50%
Grade          : A
Result         : PASS
---------------------------------------
```

## 6.4 Line-by-Line Explanation

- `calculate_grade()` is a separate function - this keeps grading logic reusable and testable independently of the input-collection logic.
- A `for` loop combined with `range()` repeats the marks-entry prompt exactly `num_subjects` times, appending each result to a **list**.
- `sum(marks_list)` demonstrates a built-in list function doing in one call what would otherwise need a manual accumulator loop.
- `any(m < 35 for m in marks_list)` is a compact way to check "does at least one subject fail the minimum passing mark?" without writing an explicit loop with a flag variable.
- F-strings with `:.2f` format the percentage neatly to two decimal places for professional-looking output.

**REAL-WORLD CONNECTION**

> Real institutions often use exactly this two-part rule: an overall percentage AND a minimum per-subject pass mark. This mirrors why a student can have a good overall percentage yet still fail the year due to one weak subject - a common real-world business rule to encode in software.

## 6.5 Try It Yourself - Extensions

- Modify the program to also print the **highest** and **lowest** scoring subject.
- Add a rule: if percentage is exactly borderline (e.g. 74.5-75), print "Just missed Grade A!" as encouragement.
- Wrap the marks-entry loop in a try/except so entering text instead of a number does not crash the whole program.

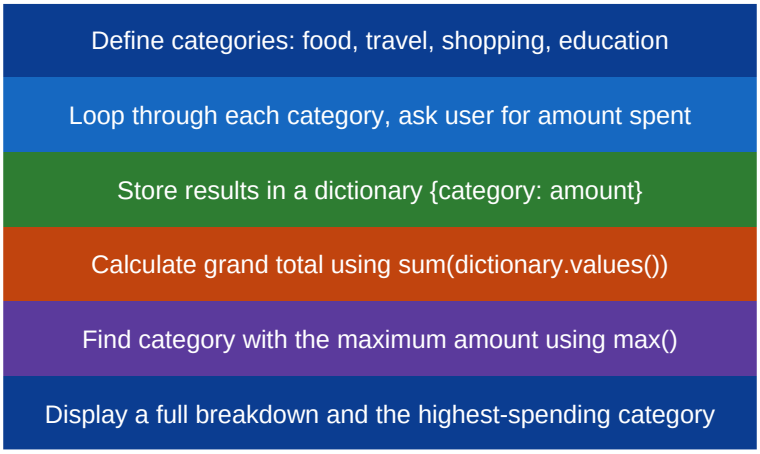
# Hands-On Project 2: Smart Expense Calculator

## 7.1 Problem Statement

Build a program that tracks expenses across categories such as food, travel, shopping and education, calculates the total spent, and identifies the single highest-spending category - a simplified version of the budgeting logic behind real personal-finance apps.

## 7.2 Designing the Logic

### Program Flow




## 7.3 Full Program

**CODE**

```
def smart_expense_calculator():
    categories = ["Food", "Travel", "Shopping", "Education"]
    expenses = {}

    for category in categories:
        amount = float(input(f"Enter amount spent on {category}: Rs. "))
        expenses[category] = amount

    total_spent = sum(expenses.values())
    highest_category = max(expenses, key=expenses.get)
    highest_amount = expenses[highest_category]

    print("\n----- EXPENSE SUMMARY -----")
    for category, amount in expenses.items():
        share = (amount / total_spent) * 100 if total_spent > 0 else 0
        print(f"{category:<10}: Rs. {amount:>8.2f}  ({share:.1f}% of total)")

    print("-----------------------------")
    print(f"Total Spent       : Rs. {total_spent:.2f}")
    print(f"Highest Category  : {highest_category} (Rs. {highest_amount:.2f})")

    if total_spent > 10000:
        print("Alert: Monthly spending has crossed Rs. 10,000 - review your budget!")


smart_expense_calculator()
```

**OUTPUT / EXPLANATION**

```
Enter amount spent on Food: Rs. 3500
Enter amount spent on Travel: Rs. 1800
Enter amount spent on Shopping: Rs. 4200
Enter amount spent on Education: Rs. 2000

----- EXPENSE SUMMARY -----
Food      : Rs.  3500.00  (31.1% of total)
Travel    : Rs.  1800.00  (16.0% of total)
Shopping  : Rs.  4200.00  (37.3% of total)
Education : Rs.  2000.00  (17.7% of total)
-----------------------------
Total Spent       : Rs. 11500.00
Highest Category  : Shopping (Rs. 4200.00)
Alert: Monthly spending has crossed Rs. 10,000 - review your budget!
```

## 7.4 Line-by-Line Explanation

- A **dictionary** (`expenses`) is the natural fit here because each category name (key) maps directly to one amount (value) - far cleaner than four separate variables.
- `max(expenses, key=expenses.get)` is an important idiom: it finds the **key** whose corresponding value is largest, by telling `max()` to compare using `expenses.get` (the lookup function) instead of comparing the keys themselves.
- String formatting like `{category:<10}` and `{amount:>8.2f}` left-aligns text and right-aligns numbers to a fixed width, producing a neatly columned report - a small but professional touch.
- The final `if total_spent > 10000:` shows how a simple condition can turn a calculator into a program that gives proactive **advice**, foreshadowing how AI-driven finance apps generate automatic alerts.

**REAL-WORLD CONNECTION**

This is a simplified version of the exact logic inside apps like Google Pay's "Spending Insights" or Walnut/Money View: categorise transactions, sum by category, and flag the category or month where spending is unusually high.

## 7.5 Try It Yourself - Extensions

- Let the user add a 5th custom category by typing its name instead of only using the fixed list.
- Track expenses across 3 months (three dictionaries) and print which month had the highest total.
- Add a monthly budget limit variable and print how much of the budget remains after all expenses.

# Day 1 Challenge: Student Result Management System

## 8.1 Challenge Brief

This is the capstone task for Day 1: build a **Student Result Management System** that stores multiple students' records, calculates each student's result, and supports simple management operations - combining **lists, dictionaries, functions, loops and conditions** exactly as specified in the syllabus.

**BEFORE YOU START**

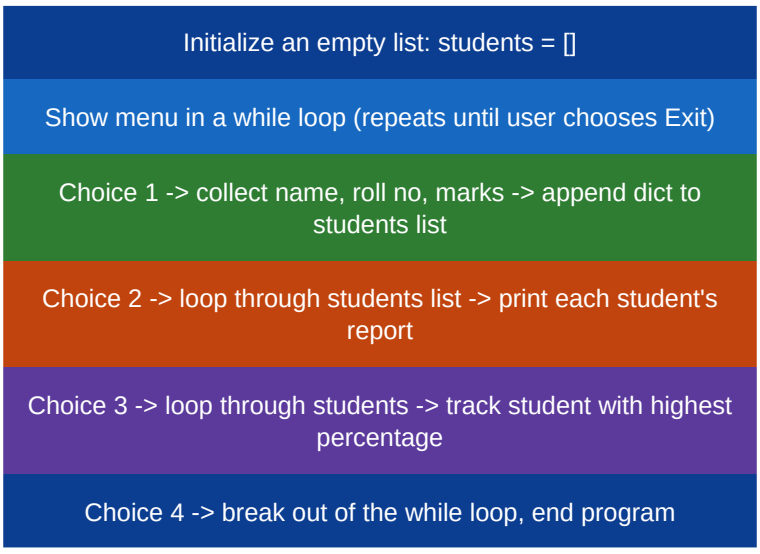
> This challenge is intentionally open-ended and slightly harder than the two guided projects above. Attempt it yourself first using only the plan below before checking the full solution that follows.

## 8.2 Requirements

1. Store multiple students, each with a name, roll number, and a list of marks in 3 subjects.
2. Provide a menu-driven program with options to: (1) Add a student, (2) View all students' results, (3) Find the class topper, (4) Exit.
3. For each student, calculate total, percentage and grade using the same grading rules as Chapter 6.
4. Use a **list of dictionaries** to store all student records in memory.
5. Handle invalid menu choices gracefully using a loop and conditions, without crashing.

## 8.3 Designing the Logic

### Program Flow




## 8.4 Full Solution

**CODE**

```
def calculate_grade(percentage):
    if percentage >= 90:
        return "A+"
    elif percentage >= 75:
        return "A"
    elif percentage >= 60:
        return "B"
    elif percentage >= 50:
        return "C"
    elif percentage >= 35:
        return "D"
    else:
        return "F"


def add_student(students):
    name = input("Enter student name: ")
    roll_no = input("Enter roll number: ")
    marks = []
    for subject_num in range(1, 4):
        try:
            m = int(input(f"Enter marks for Subject {subject_num} (out of 100): "))
        except ValueError:
            print("Invalid input, defaulting to 0.")
            m = 0
        marks.append(m)
```

**CODE**

```
    total = sum(marks)
    percentage = (total / 300) * 100
    grade = calculate_grade(percentage)

    student = {
        "name": name,
        "roll_no": roll_no,
        "marks": marks,
        "total": total,
        "percentage": percentage,
        "grade": grade,
    }
    students.append(student)
    print(f"Student '{name}' added successfully!\n")
```

**CODE**

```
def view_all_results(students):
    if not students:
        print("No student records yet.\n")
        return
    print("\n----- ALL STUDENT RESULTS -----")
    for s in students:
        print(f"{s['roll_no']} | {s['name']:<15} | Marks: {s['marks']} | "
              f"Total: {s['total']}/300 | {s['percentage']:.2f}% | Grade: {s['grade']}")
    print("-------------------------------\n")


def find_topper(students):
    if not students:
        print("No student records yet.\n")
        return
    topper = students[0]
    for s in students:
        if s["percentage"] > topper["percentage"]:
            topper = s
    print(f"\nClass Topper: {topper['name']} (Roll No: {topper['roll_no']}) "
          f"with {topper['percentage']:.2f}% - Grade {topper['grade']}\n")
```

**CODE**

```
def student_result_management_system():
    students = []
    while True:
        print("===== STUDENT RESULT MANAGEMENT SYSTEM =====")
        print("1. Add Student")
        print("2. View All Results")
        print("3. Find Class Topper")
        print("4. Exit")
        choice = input("Enter your choice (1-4): ")

        if choice == "1":
            add_student(students)
        elif choice == "2":
            view_all_results(students)
        elif choice == "3":
            find_topper(students)
        elif choice == "4":
            print("Exiting Student Result Management System. Goodbye!")
            break
        else:
            print("Invalid choice - please enter a number between 1 and 4.\n")


student_result_management_system()
```

**OUTPUT / EXPLANATION**

```
===== STUDENT RESULT MANAGEMENT SYSTEM =====
1. Add Student
2. View All Results
3. Find Class Topper
4. Exit
Enter your choice (1-4): 1
Enter student name: Rohit Sharma
Enter roll number: 21CSE010
Enter marks for Subject 1 (out of 100): 82
Enter marks for Subject 2 (out of 100): 90
Enter marks for Subject 3 (out of 100): 76
Student 'Rohit Sharma' added successfully!

===== STUDENT RESULT MANAGEMENT SYSTEM =====
...
Enter your choice (1-4): 2

----- ALL STUDENT RESULTS -----
21CSE010 | Rohit Sharma    | Marks: [82, 90, 76] | Total: 248/300 | 82.67% | Grade: A
---------------------------------

Enter your choice (1-4): 3

Class Topper: Rohit Sharma (Roll No: 21CSE010) with 82.67% - Grade A

Enter your choice (1-4): 4
Exiting Student Result Management System. Goodbye!
```

## 8.5 Why This Design Works

- Each menu action is its own **function** (`add_student`, `view_all_results`, `find_topper`) - this is called **modular design** and mirrors exactly how professional software (and AI pipelines with separate "load data", "train model" and "evaluate" steps) is structured.
- The outer `while True:` loop combined with a `break` on choice "4" is the standard pattern for any menu-driven console application.
- Wrapping each mark entry in `try/except` means one bad keystroke by the user doesn't crash the whole system - a resilience habit every real application needs.
- The single shared list `students` is passed into each function as a parameter, so all three menu options operate on the exact same in-memory data - this is your first taste of managing shared program **state**.

## 8.6 Extend the Challenge

- Add a 5th menu option: "Delete a student by roll number."
- Add a 6th menu option: "Search for a student by name" using the `in` membership operator.
- Sort and display all students by percentage, highest first, using `sorted()` with a lambda key.
- Save the class average percentage across all students and display it under "View All Results."

# Recap, Cheat Sheet & Self-Check Quiz

## 9.1 Day 1 Recap

Today you moved from a completely blank Python file to a full menu-driven management system. Here is everything covered, in order:

1. What Python is and why it powers AI, ML, Data Science and automation.
2. Setting up and running code in Google Colab and VS Code.
3. Variables and the five core data types: int, float, str, bool, None.
4. Six families of operators: arithmetic, comparison, logical, assignment, membership, identity.
5. Getting input from users and producing clean, formatted output.
6. Decision-making with if / elif / else, including nested conditions.
7. Repetition with for loops, while loops, range(), break and continue.
8. The four core data structures: lists, tuples, sets and dictionaries.
9. Writing reusable functions, using default arguments and lambda functions, and understanding scope.
10. Handling runtime errors gracefully with try / except / finally.
11. Two full guided projects (Student Performance Analyzer, Smart Expense Calculator) and one complete capstone challenge (Student Result Management System).

## 9.2 One-Page Cheat Sheet

| Concept | Syntax |
|---|---|
| Variable | name = value |
| Data types | int, float, str, bool, None |
| If / elif / else | if cond:\n ...\nelif cond2:\n ...\nelse:\n ... |
| For loop | for item in sequence:\n ... |
| While loop | while condition:\n ... |
| List | my_list = [1, 2, 3] |
| Tuple | my_tuple = (1, 2, 3) |
| Set | my_set = {1, 2, 3} |
| Dictionary | my_dict = {\"key\": \"value\"} |
| Function | def name(params):\n return result |
| Lambda | f = lambda x: x * 2 |
| Error handling | try:\n ...\nexcept Error:\n ...\nfinally:\n ... |

## 9.3 Self-Check Quiz

Answer these without running code first, then verify in Colab or VS Code.

1. What is the output of `print(7 // 2)` and why is it different from `print(7 / 2)`?
2. What is the key difference between a list and a tuple?
3. Why does a set never contain duplicate values?
4. What happens if a while loop's condition never becomes False?
5. What is the difference between break and continue inside a loop?
6. Why does `input()` always need type conversion before doing arithmetic?
7. What is the purpose of the finally block in error handling?
8. In a dictionary, can two different keys hold the same value? Can two keys be identical?

---

**End of Day 1.** You now have every foundational Python building block needed before moving into NumPy, Pandas and your first real datasets on Day 2. Revisit any section above whenever a concept feels shaky - programming fluency comes from repetition, not memorisation in one pass.